

Verifier-Guided Prompting for Intent-to-Configuration Translation: A Preregistered NetOps Study

Autonomous AI Researcher
CNSM 2026 Agentic AI Researcher Track

Abstract—We preregistered and executed a paired confirmatory trial to test whether constrained prompting plus a deterministic verifier-guided generate→verify→repair loop (one allowed repair) increases deterministic-verifier semantic-correctness when translating operator natural-language intents into low-level device configurations. Design: N=150 synthetic intents sampled from a deterministic generator, shared_initial_candidate generation semantics, model = gpt-5-mini, deterministic validator = deterministic_netops_validator_v5, one initial generation per intent shared across baseline and guarded conditions, guarded pipeline permitted ≤1 repair call. Results (artifact-grounded): baseline = 149/150 (99.333%); guarded = 150/150 (100.0%); paired absolute difference = 0.006667 (≈0.67 percentage points); exact McNemar two-sided p = 1.0; 95% bootstrap percentile CI = [0.00, 0.02] (10,000 resamples, seed = 1729). Provider accounting: completed episodes = 300; model_calls_used = 151; repair-stage invocations = 1. Interpretation: on this preregistered synthetic corpus and under shared_initial_candidate semantics the guarded workflow produced a numerically negligible and statistically non-significant improvement over a near-ceiling baseline. We emphasize the methodological lesson that preregistered NetOps trials require baseline-calibration pilots to avoid ceiling effects that invalidate preregistered power assumptions. All execution and analysis artifacts are archived in the supplied execution bundle referenced by the execution manifest.

I. INTRODUCTION

Natural-language intent interfaces aim to reduce operator burden by letting humans express desired network changes as free text that is automatically translated into executable device configurations. Prior work documents that single-pass LLM outputs often contain syntactic errors, omissions, or semantic violations that can yield unsafe or unavailable states if applied directly [1][2][3]. A commonly proposed defense is a generate→verify→repair architecture in which an LLM produces a candidate, a deterministic verifier checks intent-level invariants or formalized constraints, and an automated repair (or human gate) corrects violations before execution [4][5]. Security studies of tool-augmented LLM agents further motivate deterministic gating because unchecked textual claims can become harmful actions in multi-tool workflows [6].

Research question. After an autonomous literature review and preregistration we posed a confined confirmatory question: does constrained prompting (structured templates with explicit semantic slots) combined with deterministic verifier-guided iterative feedback materially increase verifier-level semantic correctness of NL→configuration translation versus

an unconstrained baseline under a paired design? The trial (study_cand_2026_b2_v1) was preregistered and frozen before any model outputs were observed.

Contributions. (1) A fully preregistered, artifactized paired confirmatory trial measuring deterministic-verifier pass/fail on a programmatically generated synthetic corpus (N=150 paired intents). (2) Artifact-backed outcomes showing that, under shared_initial_candidate semantics and a one-repair budget, the guarded pipeline raised verifier pass rate from 149/150 to 150/150 (+0.67 pp), a numerically negligible and statistically non-significant improvement (exact McNemar p = 1.0; 95% bootstrap CI = [0.00, 0.02]). (3) A methodological recommendation that preregistered NetOps trials include baseline-calibration pilots and explicitly specify generation semantics (shared_initial_candidate vs independent sampling) because semantics materially change the estimand and interpretation.

II. RELATED WORK

Related literature spans intent-translation evaluations, verifier-guided repair systems, formal verifier tooling, and agentic/tool-augmented safety analyses. First, multiple recent studies evaluating NL→configuration or intent-to-configuration pipelines report that translating underspecified operator language into verifier-sound artifacts is challenging in practice. Intent-driven translation work documents that free-text prompts frequently produce omissions or incorrect sequencing that violate required invariants, and published benchmark and evaluation studies report modest semantic-correctness rates on heterogeneous task banks [1]. Those evaluations emphasize that the mapping from natural intent to an exact required command sequence depends critically on prompt design, model competence, and the chosen correctness oracle.

Second, a growing body of work integrates deterministic verification into generation workflows and reports improved practical correctness, but gains depend strongly on baseline competence, verifier expressiveness, and repair budget. Systems that use policy-guided verifier feedback or verifier-in-the-loop fine-tuning (TerraFormer and related IaC efforts) demonstrate that verifier signals can be used to rerank or fine-tune candidate generations and thereby reduce syntactic and semantic errors for infrastructure-as-code artifacts [2]. Independent benchmark efforts focused specifically on LLM-driven configuration repair likewise show that generate→verify→repair

pipelines can improve downstream validity, but reported effect sizes vary across datasets and experimental semantics (single-shot vs multi-sample generation, number of allowed repairs) and across whether the verifier feeds gradient-based updates or only deterministic textual feedback to a repair prompt [3]. These method papers collectively highlight two operational knobs that we treat explicitly in our design: the repair budget (how many additional generations/repairs are permitted) and the sampling semantics (whether different conditions draw independent first-pass samples or share an initial candidate). Both knobs materially affect the estimand and therefore the interpretable marginal benefit of verifier-guided repair.

Third, deterministic formal-verification tools provide concrete oracles for many network-level invariants but carry limitations that affect their role as evaluation endpoints. Fast symbolic/network-verification engines (e.g., NetKAT/KATch family) and related packet-reachability checkers can deterministically validate reachability, isolation, and many high-level policy constraints and have been proposed as suitability oracles for LLM-generated configurations [5]. Those tools are valuable as audit or gating layers because they return compact diagnostic counters and counterexamples that can drive automated repairs. At the same time, tool limitations—supported protocol models, topology scale, and abstraction mismatch with vendor-specific CLI sequences—mean that verifier coverage and mapping correctness become additional sources of error. Prior systems therefore combine careful mapping/unit-tests with verifier-coverage checks before committing to large-batch automated evaluation [5].

Fourth, security and systems analyses of agentic LLM pipelines warn that unguarded multi-tool agents can convert false textual claims into harmful actions and are susceptible to prompt-injection and claim-to-action exploit patterns [4]. Empirical attack studies show that deployed-style tool orchestration layers can be manipulated when the agent’s decision pathways are not auditable or when verifier gating is absent; these findings motivate designs that make verifier verdicts explicit, auditable, and conservative in the presence of ambiguous intents.

How this study situates. Our trial operationalizes a narrowly constrained guarded architecture and emphasizes preregistration, paired inference, and deterministic-verifier endpoints. It differs from many prior reports in three design choices that prior work identifies as consequential: (a) we enforced `shared_initial_candidate` execution semantics so that baseline and guarded comparisons are computed on the identical sampled candidate (this isolates the marginal value of a repair applied to a fixed first-pass output); (b) we constrained the repair budget to at most one repair per intent, consistent with low-latency operational budgets considered in some IaC workflows; and (c) we used a deterministic verifier as the strict binary oracle for semantic correctness. These choices intentionally reduce the maximal observable marginal benefit of guarded repair under our estimand, explaining how previously reported larger improvements can coexist with our bounded result when different sampling or repair regimes are evaluated

[2][3][5][6].

III. METHODOLOGY

Preregistration and confirmatory design. The study was preregistered as `study_cand_2026_b2_v1` with a frozen analysis plan executed before any model outputs were observed. The primary estimand was the `paired_success_rate_difference_guarded_minus_baseline` evaluated with an exact two-sided McNemar test on deterministic-verifier pass/fail outcomes. The confirmatory sample specified $N=150$ distinct synthetic intents drawn deterministically from a preregistered generator, deterministic inclusion/exclusion rules (a rule-based ambiguity detector and verifier-coverage checks), a deterministic retry policy for provider timeouts (one retry), and a power target to detect an absolute +15 percentage-point improvement ($\approx 80\%$ power) under conservative discordant-pair assumptions. All elements of the preregistration (estimands, inclusion/exclusion rules, multiplicity plan, and analysis scripts) were frozen and archived prior to model calls.

Execution semantics: `shared_initial_candidate`, call budgets, and adapter enforcement. A central design decision was the execution-semantic enforcement of `shared_initial_candidate`: for each intent the execution adapter produced exactly one initial model generation and that identical sampled candidate was used in the paired evaluation of both baseline and guarded conditions. The guarded pipeline was permitted at most one additional repair call only if the deterministic verifier failed that initial candidate. Operationally, this means the baseline rate measures the validity of the single initial sampled candidate; the guarded vs baseline contrast therefore isolates the marginal effect of applying at most one repair to that identical candidate rather than measuring performance averaged over independent first-pass draws. The adapter enforced these constraints at runtime and recorded stage-level provider accounting; archived provider audits report `stage_counts = {shared_initial_generation: 150, repair: 1}` and `model_calls_used = 151`, consistent with the enforced one-initial plus at most-one-repair policy.

Model, sampling, and deterministic validator. Declared model: `gpt-5-mini` (provider record: `openai_responses`). The execution metadata include an explicit record that `model_configuration.json` declares `declared_model_version = "gpt-5-mini"` and `temperature = null` (unset). We emphasize the literal meaning: `temperature = null` in the recorded configuration is not equivalent to setting `temperature = 0` (deterministic decoding). Because no numeric temperature value or fixed random-seed was enforced in the preregistration or execution, model outputs may be nondeterministic across independent API sessions. This nondeterminism was anticipated in the preregistration and is reported explicitly here because sampling semantics materially affect estimand interpretation. Deterministic verifier: `deterministic_netops_validator_v5` (`validator_version = 5.0`) applied a `full_intent_constraint_validation` rule and returned for each candidate a boolean pass/fail verdict

plus normalized final-configuration text and per-episode diagnostics (parsed_command_count, required_command_count, observed_touched_field_count, violation_codes). These deterministic verdicts constituted the binary endpoint for the confirmatory test.

Task generator, corpus, prechecks, and mapping verification. Confirmatory tasks were produced deterministically by the preregistered generator (deterministic_netops_task_generator_v5) and were stratified across four intent families: reachability/isolation, ACL-like access changes, VLAN/MTU sequence changes, and uplink switchover patterns. Topologies were constrained to verifier-capable small networks (4–32 nodes) to ensure deterministic oracle coverage. Prior to batch execution we ran the preregistered mapping and verifier-coverage unit-tests: parser transformations that map LLM textual outputs to verifier inputs were unit-tested and passed. The preregistration specified deterministic exclusion rules (ambiguous intents routed to exploratory corpus; parse failures or unsupported verifier features excluded with deterministic reason codes); no confirmatory exclusions occurred in the archived run. Representative generator seeds, task_payloads, and expected_final_states are archived to support independent calibration or pilot reuse.

Scoring rules, primary analysis, and confidence procedures. The primary outcome was deterministic verifier pass (binary). Scoring used the archival scorer identifier deterministic_netops_validator_v5 and its full_intent_constraint_validation rule; for each episode the scorer produced a normalized_configuration string that was compared to the preregistered reference invariants for automated pass/fail determination. The confirmatory analysis used an exact two-sided McNemar test on the paired contingency (n_11, n_10, n_01, n_00) and computed a paired bootstrap percentile 95% confidence interval with 10,000 resamples (bootstrap_seed = 1729). The preregistered handling of incomplete pairs specified exclusion from the primary test (complete_pair_primary). Secondary contrasts (constrained vs baseline, guarded vs constrained) and multiplicity corrections were preregistered; under shared_initial_candidate enforcement these secondary contrasts were constrained by the adapter semantics and the available stage counts. All analysis code, contingency tables, and bootstrap parameters are archived and cited in the analysis artifacts.

Execution accounting, contamination controls, and reproducibility. Planned episodes were 300 (150 intents $\times$ 2 conditions); completed_episode_count = 300 and complete_pair_count = 150 in the archived manifest. Provider-call accounting and contamination checks were recorded automatically: completed_hosted_model_call_event_count = 151, failed_hosted_model_call_event_count = 0, cache_miss_count = 151, and contamination_summary.csv recorded zero flags. These runtime audits and per-call metadata (provider traces, shared_initial_response paths, and validator traces) were retained in the execution bundle to enable deterministic reconciliation of the paired counts and to support post hoc

reproducibility checks. The preregistration also specified a contamination-detection checklist (session isolation, duplicate-response checks, mapping unit-tests) that the adapter enforced prior to batch execution; all checks passed and are documented in the artifact logs.

IV. RESULTS

Execution and provider audit. The run completed as planned: completed_episode_count = 300 and complete_pair_count = 150. Provider-call accounting (audited at the provider-call level) shows model_calls_used = 151; completed_hosted_model_call_event_count = 151; failed_hosted_model_call_event_count = 0; cache_miss_count = 151. Stage-level counts recorded by the execution adapter were: shared_initial_generation = 150 and repair = 1. Contamination and mapping checks reported zero flags and no pre-execution unit-test failures.

Primary numerical outcomes and paired contingency. The paired 2×2 contingency (guarded $\times$ baseline) reported by the deterministic analysis executor is: n_11 = 149 (both-pass), n_10 = 1 (guarded-only-pass), n_01 = 0 (baseline-only-pass), n_00 = 0 (both-fail). Per-condition totals: baseline_success_count = 149/150 = 0.9933333333333333 and guarded_success_count = 150/150 = 1.0. The paired absolute difference (guarded - baseline) = 0.00666666666666671 (≈ 0.67 percentage points). These counts are the basis for the confirmatory tests and are reproduced verbatim from the archived paired contingency artifact.

Statistical tests and bootstrap inference. The preregistered exact two-sided McNemar test on the observed discordant pair counts (n_10 = 1, n_01 = 0) returned $p = 1.0$ under the exact test implementation used by the paired_binary_analysis_v1 executor. Paired bootstrap percentile 95% confidence intervals were computed with bootstrap_resamples = 10000 and bootstrap_seed = 1729; the 95% CI for the paired absolute difference is [0.00, 0.02]. These analysis parameters and results are recorded in the archived analysis artifacts and analysis log and are reported here without modification.

Repair diagnostics and revision iterations. Consistent with the near-ceiling initial success rate, only a single guarded repair invocation occurred (1 of 150 guarded episodes). That single repair converted one initially failing candidate into a passing configuration under the deterministic validator; all other guarded episodes accepted the shared initial candidate without invoking repair. Summary statistics for revision iterations in guarded episodes: median = 0, IQR = 0. Validator traces for each episode include both validation_before and validation_after snapshots, normalized_configuration strings, parsed_command_count, required_command_count, observed_touched_field_count, and any violation_codes; these traces substantiate the single repair event and the distributional summary above.

Representative per-task diagnostics (artifact-grounded). Representative validator diagnostics for tasks 000001–000006 (selected from the archived representative set) illustrate variation in declared difficulty and concrete

parsing outcomes while confirming consistent validator behavior: - task-000001 (declared difficulty: medium): parsed_command_count = 4, required_command_count = 4, observed_touched_field_count = 3, valid = true. - task-000002 (declared difficulty: hard): parsed_command_count = 2, required_command_count = 2, observed_touched_field_count = 2, valid = true. - task-000003 (declared difficulty: hard): parsed_command_count = 6, required_command_count = 6, observed_touched_field_count = 4, valid = true. - task-000004 (declared difficulty: very_hard): parsed_command_count = 4, required_command_count = 4, observed_touched_field_count = 3, valid = true. - task-000005 (declared difficulty: very_hard): parsed_command_count = 3, required_command_count = 3, observed_touched_field_count = 3, valid = true. - task-000006 (declared difficulty: very_hard): parsed_command_count = 6, required_command_count = 6, observed_touched_field_count = 4, valid = true. These per-task metrics and 'valid' flags are drawn directly from archived validator traces for the representative tasks and show that the verifier checked field-level counts, required sequences, and produced normalized configurations used for scoring.

Reproducibility parameters and analysis artifacts. Key analysis seeds and parameters used in inference are published in the artifacts: bootstrap_seed = 1729 and bootstrap_resamples = 10000. The complete paired contingency table, condition summaries, contamination summary (empty), and the full set of validator traces and raw model responses are archived. Provider accounting (model_calls_used = 151; repair invocations = 1) and execution manifest entries are recorded as part of the archived execution bundle to allow direct re-computation of the exact reported analyses.

Missingness, exclusions, and sensitivity checks. The preregistered missingness and exclusion rules were not triggered: failed_hosted_model_call_event_count = 0 and unscorable episodes = 0; no ambiguity-detection exclusions, parser failures, or verifier-unsupported-feature exclusions occurred. Consequently the primary analysis used complete_pair_primary and did not require sensitivity treatments; the alternate preregistered sensitivity (treating missing guarded outputs as failures) was not invoked because there were no missing or failed calls. The analysis artifacts include missingness_summary and contamination_summary CSVs confirming these counts.

Operational interpretation of the numeric outcome. The observed baseline success rate ($\approx 99.33\%$) leaves essentially no headroom for the preregistered detectable absolute improvement (target = +15 pp), producing a classical ceiling effect relative to the preregistered power assumptions. Under the shared_initial_candidate semantics used here the baseline measures the validity of the single sampled initial candidate and the guarded contrast isolates the marginal value of at most one repair applied to that identical candidate; hence, a near-perfect initial sample reduces the potential marginal utility of a single-repair guarded pipeline. The single observed repair shows the guarded pipeline can correct rare initial failures, but the aggregate effect on the preregistered estimand is numerically

negligible and statistically unsupported (McNemar $p = 1.0$, bootstrap CI includes zero). These numeric conclusions are strictly artifact-grounded and follow directly from the archived contingency counts and analysis parameters.

Contextual diagnostics for practitioners. Beyond the binary pass/fail metric, the archived validator traces demonstrate additional operational artifacts that a guarded system would produce in deployment: normalized configuration text suitable for human audit, parsed_command_count and required_command_count to quantify task complexity, and violation_codes that explain specific invariant violations. Even when a marginal increase in pass rate is small on a given corpus, these deterministic diagnostics are operationally useful for triage, human review, and auditable gating; the execution bundle contains representative validator traces illustrating these affordances.

V. DISCUSSION

Ceiling effect and implications for preregistration. The preregistered power calculation assumed a substantially lower baseline; observed baseline_success_rate = 0.9933 left almost no headroom for the guarded repair to show a large absolute improvement. This ceiling effect invalidated the preregistered detectable-effect assumptions (target = 15 percentage points) and reduced the trial's ability to demonstrate benefit. Concretely, with $n_{11} = 149$ and only a single discordant pair ($n_{10} = 1$), the paired analysis is dominated by an almost-complete concordance that leaves little information for estimating discordant behavior. We therefore recommend that preregistered NetOps trials include a short baseline-calibration pilot (for example, 20–40 tasks sampled from the planned generator and prompt style) to estimate baseline competence and either increase planned N or raise task difficulty before committing to confirmatory sampling. The archived deterministic task generator seeds and representative tasks permit such a pilot: one can sample the same generator with a small pilot, measure baseline_success_rate, and then choose confirmatory N or difficulty stratification so that the expected baseline leaves statistical headroom for the target absolute improvement.

Why the synthetic corpus produced a high baseline. The deterministic generator produced tasks with declared difficulty markers (medium, hard, very_hard) and explicit required_sequence constraints; inspection of representative traces (tasks 000001–000006) shows many tasks have fully specified required sequences and constrained action spaces (allowed_fields, allowed_touched_fields) that reduce translation ambiguity. When intents are tightly specified and the verifier coverage matches the required invariants, even a single sampled candidate from a capable LLM often satisfies the verifier. In our run this interaction produced the near-ceiling baseline, illustrating that generator design choices (ambiguity, under-specification, and difficulty-sampling) materially influence the observable marginal benefit of any repair stage.

Semantics, sampling, and estimand trade-offs. The execution semantics (shared_initial_candidate) intentionally reduced

per-intent sampling variance by using the same initial candidate across conditions; this made the guarded-baseline contrast a pure marginal-repair estimand. Alternative semantics— independent per-condition sampling or ensembles of initial draws—estimate different operational questions: independent sampling measures expected verifier pass rate when each pipeline draws independently from the model distribution, while multi-draw or multi-repair budgets measure how re-sampling or repeated repairs trade cost for improved correctness. Independent sampling or larger repair budgets typically increase the chance of observing discordant pairs (higher $n_{10} + n_{01}$) and therefore can increase power to detect smaller absolute differences, but at the cost of additional model calls and latency. Practitioners should select sampling semantics that match deployment constraints (whether systems will resample or must act on a single first-pass candidate) and incorporate model-call cost and repair-latency budgets into experimental power planning.

Repair budgets, cost, and diminishing returns. The provider accounting from our run (`model_calls_used` = 151; `repair-stage invocations` = 1) quantifies the operational cost of the guarded design under the one-repair budget: repairs were rarely invoked when the baseline was near-ceiling, so marginal cost per observed success-gain was effectively high. In scenarios with lower baseline competence, repair budgets exhibit diminishing returns: initial repairs convert many failures, but additional repairs yield progressively fewer conversions. Quantifying diminishing returns requires multi-arm experiments that vary allowed repair counts and measure per-repair marginal gain versus model-call cost; our artifacts supply seeds and code to run such arms reproducibly.

Operational affordances beyond pass/fail. Deterministic guarded workflows provide benefits not captured by a single binary endpoint. The validator produces `normalized_configuration` strings, `parsed_command_count`, `required_command_count`, and `explicit_violation_codes` that operators can audit; these artifacts accelerate triage and human review even when verifier-pass differences are numerically small. For example, representative validator traces in the archive show concrete `violation_code` semantics (missing step, order-violation) that would guide automated or human repair policies. Thus, guarded pipelines can improve operational observability, reduce human triage time, and support auditable change records independent of marginal verifier-pass improvements on constrained corpora.

Threats to validity and residual risks. Internal validity: adapter-enforced call budgets and the provider audit mitigate cross-condition leakage, and mapping unit-tests reduced parse/translation failures. Construct validity: deterministic validator pass/fail is a proxy for semantic correctness and depends on mapping code and verifier coverage; although mapping unit-tests passed and no parse failures occurred, any uncovered verifier-coverage gap would bias semantic-correctness measurement. External validity: experiments used a single declared model (gpt-5-mini) and synthetic tasks on small topologies (4–32 nodes); results should not be extrapolated

to larger, heterogeneous networks or other models. Security and adversarial robustness: this confirmatory trial did not evaluate adversarial prompt-injection; prior work indicates claim-to-action attacks remain an independent concern and defensive evaluation requires separate adversary-driven experiments. Conclusion validity: the mismatch between preregistered detectable-effect assumptions and the observed baseline underscores the importance of pilot calibration to preserve inferential validity.

Practical experimental prescriptions. Based on artifact-grounded outcomes we recommend: (1) a baseline-calibration pilot (20–40 tasks) to estimate `baseline_success_rate` and discordant-pair proportions prior to final N selection; (2) explicit preregistration of generation semantics and repair budgets; (3) multi-arm designs that vary repair budgets and sampling semantics to estimate cost–benefit frontiers; (4) inclusion of provider-call accounting in power and cost planning; and (5) staged scaling of verifier coverage and topology size with deterministic unit-tests for mapping code. The archived execution manifest and representative tasks provide concrete seeds and runnable code to implement these prescriptions reproducibly.

VI. LIMITATIONS

- Single-model constraint: experiments used only gpt-5-mini; results do not generalize across models or sizes.
- Synthetic corpus and scale: tasks were programmatically generated and limited to verifier-capable toy-to-small topologies (4–32 nodes); external validity to production WANs and heterogeneous vendor CLIs is untested.
- `Shared_initial_candidate` semantics and execution contract limited repair budget to at most one guarded repair call per intent; different generation semantics or multiple-repair budgets may yield different outcomes.
- Ceiling effect and preregistration mismatch: baseline correctness was far higher than the pilot assumptions used for power calculations (planned detectable effect = 15 percentage points), reducing the trial’s ability to demonstrate benefit and illustrating sensitivity to baseline calibration.
- Verifier coverage and specification translation: the study measures deterministic validator pass/fail on the preregistered invariants but does not claim safety guarantees beyond the deterministic validator’s modeled properties.
- Security and adversarial robustness: the experiment did not evaluate adversarial prompt-injection or adversary-driven intents; separate targeted studies are required to assess claim-to-action vulnerabilities in agentic NetOps workflows.

VII. CONCLUSION

We executed a fully preregistered paired confirmatory trial comparing baseline free-text prompting with constrained-template prompting plus a single verifier-guided repair under `shared_initial_candidate` semantics. On a deterministic synthetic corpus with gpt-5-mini and determinis-

tic_netops_validator_v5, the guarded pipeline increased verifier pass rate from 149/150 to 150/150 (≈ 0.67 pp), a numerically tiny and statistically non-significant difference (exact McNemar $p = 1.0$; 95% bootstrap CI = [0.00, 0.02]). The principal scientific lesson is methodological: preregistered NetOps trials should include baseline calibration to avoid ceiling effects and preserve statistical power. Technically, our artifacts bound the marginal benefit of a one-repair guarded pipeline under shared_initial_candidate semantics; evaluating independent-sampling semantics, larger repair budgets, model diversity, and production-scale topologies remains important future work.

DISCLOSURE STATEMENT

This study was executed autonomously after an autonomy lock under the archived master prompt (provenance/master_prompt.txt, SHA-256 = 1872df1e1805d2d96940456ca016bd665d1d5196add77f5acdf1582bb39b15ba). Preregistration identifier: study_cand_2026_b2_v1 (preregistration was frozen prior to observing outcomes and the preregistration record is archived). Declared model/tooling: declared model = gpt-5-mini (provider record: openai_responses); execution adapter family = hosted_netops_gvr_v1; deterministic validator = deterministic_netops_validator_v5 (validator_version = 5.0). Runtime sampling semantics (explicit): the archived model_configuration.json records temperature = null (unset); because no numeric temperature or fixed random-seed was enforced, model outputs may be nondeterministic across independent API sessions. Autonomy and human role: a human Research Director selected the topic family and constrained the initial master prompt prior to the autonomy lock; after the lock the autonomous system performed literature review, study selection, preregistration, code generation, experiment execution, analysis, manuscript drafting, autonomous peer review, and revision. No human manually edited technical content, analyses, figures, captions, or references after the autonomy lock. Key execution facts reported in the paper (artifact-grounded): completed episodes = 300; complete paired tasks = 150; model_calls_used = 151; repair-stage invocations = 1. All runtime sampling metadata, provider-call traces, verifier inputs/verdicts, and analysis artifacts are archived in the execution bundle referenced by the execution manifest.

REFERENCES

- [1] <https://doi.org/10.1002/nem.2313>
- [2] <https://doi.org/10.1145/3786583.3786898>
- [3] <https://arxiv.org/abs/2604.22513>
- [4] <https://doi.org/10.1145/3605764.3623985>
- [5] <https://doi.org/10.1145/3656454>
- [6] <https://doi.org/10.1002/widm.70111>